leveraging TypeScript tooling to enforce type policies and streamline builds. Mastery of these concepts is essential for roles where code quality, scalability, and developer experience are paramount.

In technical interviews, candidates like Jane can stand out by demonstrating expertise in TypeScript's static typing, generics, experimental decorators, and compatibility with modern ECMAScript proposals—such as optional chaining. This reflects a deep understanding of TypeScript's role in reducing runtime errors and improving code maintainability. Discussing emerging trends and advancements in TypeScript further highlights a candidate's depth of knowledge.

Table 3.7 summarizes Jane's review of the essential concepts and tools for mastering type safety and applying them effectively in real-world scenarios.

Table 3.7 Key features of type safety and tools

Topic	Fundamentals	Applied skills	Advanced
Type narrowing	Use `typeof`, `instanceof`, and control flow analysis to narrow types.	Create user-defined type guards, handle discriminated unions.	Architect safe APIs with exhaustive type checks, leverage never for unreachable code.
Type assertions & casting	Use `as` and angle-bracket syntax for casting.	Refactor code to reduce reliance on assertions and avoid unsafe casts.	Audit large codebases for unsafe assertions; enforce strict type safety policies.
Strictness & config	Enable `strict` mode, explain key compiler config options.	Configure strictness for projects, resolve common strictness errors.	Architect enterprise-scale codebases with strictest settings, automate config validation across teams.
JavaScript	Use `.d.ts` files,	Write custom	Integrate with